ARCANE VAULT

Project Implementation Documentation

D&D Character Management Web Application

Tristan De La Borda

BCLC24

2026

Table of Contents

Contents

1. Project overview

1.1 Introduction

This is the implementation guide for my project submission for my final year in finishing my BTS Cloud Computing diploma, for this project the goal was to showcase and highlight techniques and skills that I have learned for the duration of my studies. The project revolves around a hybrid cloud deployment of a web application which maintains proper fault tolerance, scalability and high availability. For the web app (Web application) the design is simple as it is not to pull the focus of the attention away from the infrastructure behind it. The web app is a simple website that can be used as a D&D (Dungeons & Dragons) character creation tool, alongside a community pool for friends alike to share and export their characters.

1.2 Project goals and objectives

To go over the main goals this project highlights, firstly, the automated infrastructure deployment of the entire web app and any such resources as contributed to it. Additionally, The storage of data along a hybrid cloud infrastructure for lowered cost of operations in database storage. Automated provisioning and deployment of resources across the hybrid cloud environment. Lastly maintaining a baseline infrastructure that supports high availability, scalability and basic security.

1.3 Scope

In scope

- Hybrid Cloud deployment
- Azure AKS on Azure
- Database VM on Proxmox
- Automated infrastructure Provisioning & Deployment
- Containerized application with docker
- Azure Container registry for images
- Entra ID authentication application
- Azure RBAC for AKS
- Network segmentation
- PostgresSQL Database
- Wireguard VPN
- VMs Uncomplicated Firewall
- Redis Caching
- Autoscaling HPA
- Ansible Vault
- K8s Secrets
- Nginx ingress

- TLS 1.3

Scope Changes

- MFA: Due to time constraints MFA on Entra ID was omitted from the final product submission.
- WAF: Also, due to time constraints less valued asset deployments were dropped from scope.

1.4 Technical stack summary

Frontend

- React
- Ingress
- Nginx
- CSS

Backend

- Node.js
- PostgresSQL
- Rest API
- Redis

Azure Infrastructure

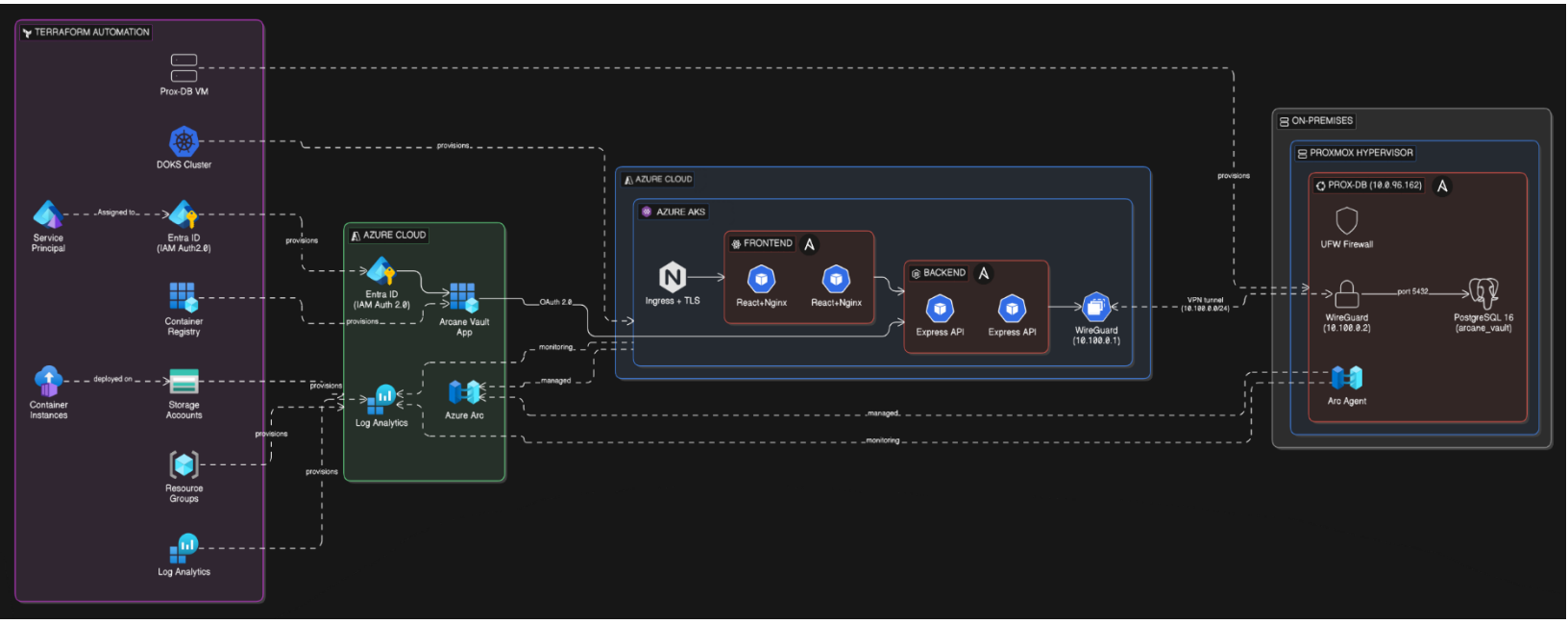
- AKS Cluster
- Entra ID RBAC
- Azure Container Registry
- Log Analytics
- Azure Monitor
- Ingress Controller
- Cert Manager
- Azure Storage

- Azure Container
- Wireguard

Proxmox Infrastructure

- Ubuntu VMs
- Wireguard
- UFW on all VMs
- Service Principal
- Azure Arc Agent
- PostgresSQL

1.5 Architecture Diagram



2. Infrastructure provisioning

2.1 Terraform: Azure resource provisioning

For the Terraform we I have a main.tf file within this file it deploys the azure infrastructure. For all the azure resources I have for the azure deployments I used the Hashicorp/Azuread and Azurerm providers for their Terraform example scripts to setting up resource groups, azure container registry, AKS cluster, and Entra id for the application and service registration.

For the resource group in the example code all I had to add was the name, location and tags in correspondence with my azure policy tags.

```
# ── Resource Group ──────────────────────────────────────

resource "azurerm_resource_group" "main" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}
```

For the Entra ID needed to access tokens for end to end connections between my on prem VMs and azure arc resources onboarding.

```
# ═══════════════════════════════════════════════════════════
# AZURE ENTRA ID APP REGISTRATION
# ═══════════════════════════════════════════════════════════

resource "azuread_application" "arcane_vault" {
  display_name = "Arcane Vault"
  owners       = [data.azuread_client_config.current.object_id]

  web {
    redirect_uris = var.redirect_uris
    implicit_grant {
      access_token_issuance_enabled = true
      id_token_issuance_enabled     = true
    }
  }

  api {
    requested_access_token_version = 2
  }

  required_resource_access {
    resource_app_id = "00000003-0000-0000-c000-000000000000" # Microsoft Graph

    resource_access {
      id   = "e1fe6dd8-ba31-4d61-89e7-88639da4683d" # User.Read
      type = "Scope"
    }
    resource_access {
      id   = "37f7f235-527c-4136-accd-4a02d197296e" # openid
      type = "Scope"
    }
    resource_access {
      id   = "64a6cdd6-aab1-4aaf-94b8-3cc8405e90d0" # email
      type = "Scope"
    }
    resource_access {
      id   = "14dad69e-099b-42c9-810b-d002981feec1" # profile
      type = "Scope"
    }
  }
}
```

Additionally for the registration I would need azure arc service principle permissions so that I wouldn't have to manually login to onboard the machines.

```
# ═══════════════════════════════════════════════════════════
# AZURE ARC SERVICE PERMISSIONS
# ═══════════════════════════════════════════════════════════

data "azuread_service_principal" "arc" {
  client_id = var.arc_sp_id
}

resource "azurerm_role_assignment" "arc_connected_machine_onboarding" {
  principal_id         = data.azuread_service_principal.arc.object_id
  role_definition_name = "Azure Connected Machine Onboarding"
  scope                = azurerm_resource_group.main.id
}
```

For the Azure Kubernetes Cluster I needed to specify the specifications of the machine and details on the configuration for the AKS cluster.

```
# ═══════════════════════════════════════════════════════
# AKS CLUSTER
# ═══════════════════════════════════════════════════════

resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-b2clc-deltr691"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = var.project_name
  kubernetes_version  = var.kubernetes_version
  tags                = var.tags

  default_node_pool {
    name                = "default"
    vm_size             = var.aks_vm_size
    os_disk_size_gb     = 50
    auto_scaling_enabled = true
    min_count           = 1
    max_count           = var.aks_max_node_count
  }

  identity {
    type = "SystemAssigned"
  }

  network_profile {
    network_plugin    = "azure"
    load_balancer_sku = "standard"
    service_cidr      = "172.16.0.0/16"
    dns_service_ip    = "172.16.0.10"
  }

  azure_active_directory_role_based_access_control {
    azure_rbac_enabled = true
    tenant_id          = data.azurerm_client_config.current.tenant_id
  }
}
```

For the azure container registry I needed to configure to push my docker build images to secure proprietary company details baked into the image build.

```
# ═══════════════════════════════════════════════════════
# AZURE CONTAINER REGISTRY
# ═══════════════════════════════════════════════════════

resource "azurerm_container_registry" "acr" {
  name                = replace("${var.project_name}acr", "-", "")
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  sku                 = "Basic"
  admin_enabled       = true
  tags                = var.tags
}

resource "azurerm_role_assignment" "aks_acr_pull" {
  principal_id                     = azurerm_kubernetes_cluster.aks.kubelet_identity[0].object_id
  role_definition_name             = "AcrPull"
  scope                            = azurerm_container_registry.acr.id
  skip_service_principal_aad_check = true
}
```

Then for current logging and future VM azure arc onboard metrics I needed to create a azure log analytics workspace.

```
# ═══════════════════════════════════════════════════════════
# LOG ANALYTICS
# ═══════════════════════════════════════════════════════════

resource "azurerm_log_analytics_workspace" "main" {
  name                = "${var.project_name}-logs"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = var.tags
}
```

And lastly I need to create a one-time azure role to assign myself admin cluster permissions to manage the Kubernetes cluster.

```
# ── AKS RBAC ──────────────────────────────────────────

resource "azurerm_role_assignment" "aks_cluster_admin" {
  principal_id         = data.azurerm_client_config.current.object_id
  role_definition_name = "Azure Kubernetes Service RBAC Cluster Admin"
  scope                = azurerm_kubernetes_cluster.aks.id
}
```

2.2 Terraform: Proxmox VM provisioning

For the terraform segment that relates to the deployment of the Wireguard Vm and the PostgresSQL DB, I used the bgp/proxmox provider for their more update configs.

For the initial configuration of the Wireguard VMs I used a template that is present on the Proxmox server with its corresponding ID I used this along with the example code from the documentation in the provider page to assign and clone that template with the fields I have assigned in terraform.

```
# ── VM 1: WireGuard Gateway ──────────────────────────────
resource "proxmox_virtual_environment_vm" "wg_gateway" {
  name      = "arcane-vault-wg"
  node_name = var.proxmox_node
  vm_id     = var.proxmox_vm_id_wg

  clone {
    vm_id = var.proxmox_vm_template_id
    full  = true
  }

  agent {
    enabled = true
  }

  stop_on_destroy = true

  cpu {
    cores = var.proxmox_vm_cores_wg
  }

  memory {
    dedicated = var.proxmox_vm_memory_wg
  }

  disk {
    datastore_id = "local-lvm"
    interface    = "scsi0"
    size         = var.proxmox_vm_disk_size_wg
  }

  network_device {
    bridge = var.proxmox_vm_bridge
  }

  initialization {
    ip_config {
      ipv4 {
        address = var.proxmox_vm_ip_wg
        gateway = var.proxmox_vm_gateway
      }
    }

    user_account {
      username = var.proxmox_vm_user
      keys     = [var.proxmox_vm_ssh_public_key]
    }
  }
```

And for the Postgres SQL server here is the configuration script on Terraform.

```
# ── VM 2: PostgreSQL Database ─────────────────────────
resource "proxmox_virtual_environment_vm" "db_server" {
  name      = "arcane-vault-db"
  node_name = var.proxmox_node
  vm_id     = var.proxmox_vm_id_db

  clone {
    vm_id = var.proxmox_vm_template_id
    full  = true
  }

  agent {
    enabled = true
  }

  stop_on_destroy = true

  cpu {
    cores = var.proxmox_vm_cores_db
  }

  memory {
    dedicated = var.proxmox_vm_memory_db
  }

  disk {
    datastore_id = "local-lvm"
    interface    = "scsi0"
    size         = var.proxmox_vm_disk_size_db
  }

  network_device {
    bridge = var.proxmox_vm_bridge
  }

  initialization {
    ip_config {
      ipv4 {
        address = var.proxmox_vm_ip_db
        gateway = var.proxmox_vm_gateway
      }
    }

    user_account {
      username = var.proxmox_vm_user
      keys     = [var.proxmox_vm_ssh_public_key]
    }
  }
```

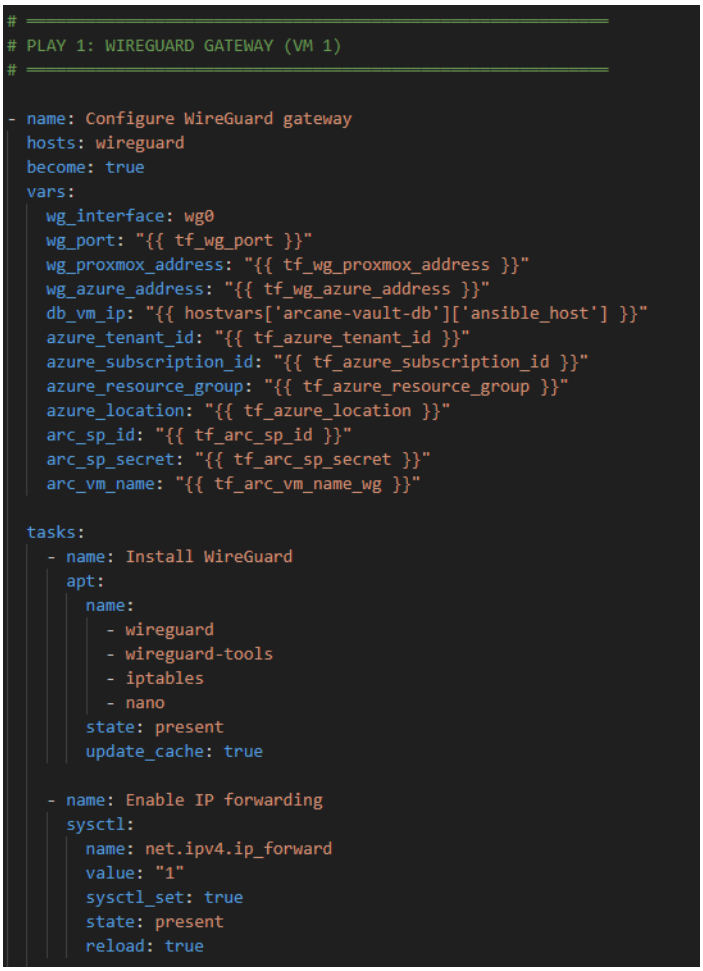
2.3 Ansible: on-premises configuration

2.3.1 WireGuard gateway (VM 1)

Once the terraform script has run and deployed the resources needed, the Ansible script is here to configure all the files and attributes on the VMs. This script will install Wireguard and Postgres on the machines and add the tables and passwords to the database and configure the wire guard settings to route IPs to and from the azure client to the database. The ansible will also configure a basic firewall that will only allow ssh access from the

management node and block all access to other IPs other than the one for wire guard VM.

For the first section we install wireguard onto the VM and assign the Wireguard configs.



```yaml
# ============================================================
# PLAY 1: WIREGUARD GATEWAY (VM 1)
# ============================================================

- name: Configure WireGuard gateway
  hosts: wireguard
  become: true
  vars:
    wg_interface: wg0
    wg_port: "{{ tf_wg_port }}"
    wg_proxmox_address: "{{ tf_wg_proxmox_address }}"
    wg_azure_address: "{{ tf_wg_azure_address }}"
    db_vm_ip: "{{ hostvars['arcane-vault-db']['ansible_host'] }}"
    azure_tenant_id: "{{ tf_azure_tenant_id }}"
    azure_subscription_id: "{{ tf_azure_subscription_id }}"
    azure_resource_group: "{{ tf_azure_resource_group }}"
    azure_location: "{{ tf_azure_location }}"
    arc_sp_id: "{{ tf_arc_sp_id }}"
    arc_sp_secret: "{{ tf_arc_sp_secret }}"
    arc_vm_name: "{{ tf_arc_vm_name_wg }}"

  tasks:
    - name: Install WireGuard
      apt:
        name:
          - wireguard
          - wireguard-tools
          - iptables
          - nano
        state: present
        update_cache: true

    - name: Enable IP forwarding
      sysctl:
        name: net.ipv4.ip_forward
        value: "1"
        sysctl_set: true
        state: present
        reload: true
```

Then transfer and assign all the public and private keys for wireguard and proxmox.

```
- name: Check if WireGuard private key exists
  stat:
    path: /etc/wireguard/private.key
  register: wg_key_check

- name: Generate WireGuard private key
  shell: wg genkey > /etc/wireguard/private.key
  args:
    creates: /etc/wireguard/private.key
  when: not wg_key_check.stat.exists

- name: Set private key permissions
  file:
    path: /etc/wireguard/private.key
    mode: "0600"
    owner: root
    group: root

- name: Generate WireGuard public key
  shell: cat /etc/wireguard/private.key | wg pubkey > /etc/wireguard/public.key
  args:
    creates: /etc/wireguard/public.key

- name: Read Proxmox private key
  slurp:
    src: /etc/wireguard/private.key
  register: wg_proxmox_private_key

- name: Read Proxmox public key
  slurp:
    src: /etc/wireguard/public.key
  register: wg_proxmox_public_key

- name: Display WireGuard public key
  debug:
    msg: >
      WIREGUARD PUBLIC KEY: {{ wg_proxmox_public_key.content | b64decode | trim }}
      — You need this key for the Azure-side WireGuard peer config.
      Save it and add it to the AKS WireGuard DaemonSet.
```

Lastly generate the wg0 profile and apply it to wireguard configs so that it connects to the node on azure.

```
- name: Create WireGuard config
  template:
    src: templates/wg0.conf.j2
    dest: /etc/wireguard/wg0.conf
    mode: "0600"
    owner: root
    group: root
  notify: restart wireguard

- name: Enable and start WireGuard
  systemd:
    name: "wg-quick@wg0"
    state: started
    enabled: true
```

Then for both this VM and the database VM ansible will configure the firewall.

```
# ── Firewall ──────────────────────────────────

- name: Install UFW
  apt:
    name: ufw
    state: present

- name: Allow SSH
  ufw:
    rule: allow
    port: "22"
    proto: tcp

- name: Allow WireGuard
  ufw:
    rule: allow
    port: "{{ wg_port }}"
    proto: udp

- name: Allow forwarding to DB VM on PostgreSQL port
  ufw:
    rule: allow
    route: true
    port: "5432"
    proto: tcp
    from_ip: "10.100.0.0/24"
    to_ip: "{{ db_vm_ip }}"

- name: Enable UFW
  ufw:
    state: enabled
    default: deny
```

2.3.2 PostgreSQL database (VM 2)

For the second VM running on premise we have my Postgres SQL database server, all of which the deployment is also done via terraform in this section in my main.tf file

```yaml
# ============================================================
# PLAY 2: POSTGRESQL DATABASE (VM 2)
# ============================================================

- name: Configure PostgreSQL database server
  hosts: database
  become: true
  vars:
    db_name: arcane_vault
    db_user: arcane_admin
    db_password: "{{ tf_db_password }}"
    pg_version: "16"
    wg_vm_ip: "{{ hostvars['arcane-vault-wg']['ansible_host'] }}"
    azure_tenant_id: "{{ tf_azure_tenant_id }}"
    azure_subscription_id: "{{ tf_azure_subscription_id }}"
    azure_resource_group: "{{ tf_azure_resource_group }}"
    azure_location: "{{ tf_azure_location }}"
    arc_sp_id: "{{ tf_arc_sp_id }}"
    arc_sp_secret: "{{ tf_arc_sp_secret }}"
    arc_vm_name: "{{ tf_arc_vm_name_db }}"

  tasks:
    - name: Install PostgreSQL and dependencies
      apt:
        name:
          - "postgresql-{{ pg_version }}"
          - "postgresql-client-{{ pg_version }}"
          - python3-psycopg2
          - acl
          - nano
        state: present
        update_cache: true

    - name: Ensure PostgreSQL is running
      systemd:
        name: postgresql
        state: started
        enabled: true

    - name: Create database
      become_user: postgres
      community.postgresql.postgresql_db:
        name: "{{ db_name }}"
        encoding: UTF-8
        state: present
```

This defines all the packages the ansible will install on the host VM including the Postgres SQL server and setup the database.

```
- name: Grant all privileges on database
  become_user: postgres
  community.postgresql.postgresql_privs:
    database: "{{ db_name }}"
    privs: ALL
    type: database
    role: "{{ db_user }}"

- name: Grant schema privileges
  become_user: postgres
  community.postgresql.postgresql_privs:
    database: "{{ db_name }}"
    privs: ALL
    type: schema
    objs: public
    role: "{{ db_user }}"

- name: Configure PostgreSQL to listen on all interfaces
  lineinfile:
    path: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf"
    regexp: "^#?listen_addresses"
    line: "listen_addresses = '*'"
  notify: restart postgresql

- name: Allow connections from WireGuard VM
  lineinfile:
    path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
    line: "host    {{ db_name }}    {{ db_user }}    {{ wg_vm_ip }}/32    scram-sha-256"
  notify: restart postgresql

- name: Allow connections from WireGuard tunnel subnet
  lineinfile:
    path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
    line: "host    {{ db_name }}    {{ db_user }}    10.100.0.0/24    scram-sha-256"
  notify: restart postgresql
```

Then furthermore the ansible will set up the database tables, schema, and listen to all interfaces for connections through the Wireguard tunnel. Lastly, just like before, Ansible will also set up the firewall rules allowing access from only the Wireguard tunnel and ssh from the management node and Wireguard VM.

2.3.3 Azure arc Onboarding both VMs

This section of the ansible will be used after onboard both the VMs to the azure arc this way we keep everything centralized.

```
# ── Azure Arc Onboarding ──────────────────────

- name: Install Azure Arc agent prerequisites
  apt:
    name:
      - curl
      - apt-transport-https
      - lsb-release
      - gnupg
    state: present

- name: Download Azure Arc connected machine agent
  get_url:
    url: https://aka.ms/azcmagent
    dest: /tmp/install_linux_azcmagent.sh
    mode: "0755"

- name: Install Azure Arc agent
  command: bash /tmp/install_linux_azcmagent.sh
  args:
    creates: /usr/bin/azcmagent

- name: Check Azure Arc connection status
  command: azcmagent show
  register: arc_status
  changed_when: false
  failed_when: false
```

Firstly it installs the azure arc agent on the host VM then it will run the end to end connection using the azure arc parameters passed from the variables to pull the service principal connection without needed manual intervention.

```
- name: Connect to Azure Arc (service principal — non-interactive)
  command: >
    azcmagent connect
    --resource-group {{ azure_resource_group }}
    --tenant-id {{ azure_tenant_id }}
    --location {{ azure_location }}
    --subscription-id {{ azure_subscription_id }}
    --resource-name {{ arc_vm_name }}
    --tags "project=arcane-vault,role=database,Owner\:=deltr691,Team\:=BCLC24"
    --service-principal-id {{ arc_sp_id }}
    --service-principal-secret {{ arc_sp_secret }}
  register: arc_connect
  when: "'Connected' not in arc_status.stdout"
  changed_when: arc_connect.rc == 0
  failed_when: arc_connect.rc != 0 and 'already' not in (arc_connect.stderr | default(''))
```

2.4 Automated deployment pipeline

Over the entire project the deployment pipeline of the resources goes as follows.

Cd ~/Arcanevault/infra/start

From this directory you can run the “Terraform apply” command.

This will set up the baseline azure resources including the storage accounts container where the next terraform script will store its tfstate file in.

```
Cd ~/Infra/Terraform
```

From this directory we need to first import the tfstate resources of the first terraform script so that they are unified and managed by the second deployment.

```
terraform import azurerm_resource_group.main /subscriptions/720f1b41-2422-4013-a9ce-1ff53e7624cc/resourceGroups/rg-b2clc-deltr-core
```

Then you can follow that up by another “Terraform apply” which will create all the main.tf script resources.

```
Cd ~/infra/ansible
```

From this directory to start we would be ideal to re-login to the azure cli just as a precautionary measure.

Get credentials after Terraform apply

```
az aks get-credentials --resource-group rg-b2clc-deltr-core --name aks-b2clc-deltr691 --overwrite-existing
```

Re-apply cluster admin role to me for cluster management

```
AKS_ID=$(az aks show --resource-group rg-b2clc-deltr-core --name aks-b2clc-deltr691 --query id -o tsv)

MY_ID=$(az ad signed-in-user show --query id -o tsv)

az role assignment create --assignee "$MY_ID" --role "Azure Kubernetes Service RBAC Cluster Admin" --scope "$AKS_ID"
```

Then we continue with ansible.

```
Ansible site.yml –ask-become-pass –ask-vault-pass
```

This should configure everything in the site.yml ansible script.
This will cover everything from the deployment and provisioning of the terraform and ansible resources.

3. Containerisation and orchestration

3.1 Docker images

3.1.1 Frontend Dockerfile

In the docker folder I have two docker files, one for the backend and one for the frontend, here we cover the front end docker file. This is the image built for the alpine OS it will install nginx onto the image and copy a config file for the nginx into the container.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY public/ ./public/
COPY src/ ./src/
ARG REACT_APP_API_URL
ARG REACT_APP_AZURE_CLIENT_ID
ARG REACT_APP_AZURE_TENANT_ID
ARG REACT_APP_AZURE_REDIRECT_URI
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

3.1.2 Backend Dockerfile

In the dockerfile backend here the image is being built ona node:alpine image this already has most of what we need already on it so all we do is setup some directories for use and install npm.

```
FROM node:20-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
COPY src/ ./src/
```

3.2 Azure Container Registry

For the azure container registry first we I build the images using the following commands:

cd ~/prod2/arcane-vault

docker build -t arcanevaultacr.azurecr.io/frontend:latest -f infra/docker/Dockerfile.frontend ./frontend

docker build -t arcanevaultacr.azurecr.io/backend:latest -f infra/docker/Dockerfile.backend ./backend

Then after this building finishes we will push the images to the Azure container registry.

az acr login --name arcanevaultacr

Relogin to the az cli just as a precautionary measure.

```
docker push arcanevaultacr.azurecr.io/frontend:latest

docker push arcanevaultacr.azurecr.io/backend:latest
```

3.3 Kubernetes (AKS) deployment

3.3.1 Cluster configuration

For the Kubernetes cluster it was already deployed by the Terraform but let me run down some specifications.

Node Pool VMs	Standard_D6v3 vCPUs
Replicas	1 min, 4 Max
Azure CNI	Azure CNI configured network
Entra ID	Tennant ID for log-in

3.3.2 Backend deployment

For the backend deployment of the Kubernetes in my manifest it has the replicas that it can make for the backend pod, that will pull from the azure container registry and allocate the number of resources specified. Along with the connection to the frontend and api ports.

```yaml
spec:
  containers:
    - name: backend
      image: arcanevaultacr.azurecr.io/backend:latest
      ports:
        - containerPort: 5000
      envFrom:
        - configMapRef:
            name: arcane-vault-config
        - secretRef:
            name: arcane-vault-secrets
      resources:
        requests:
          cpu: 100m
          memory: 128Mi
        limits:
          cpu: 500m
          memory: 512Mi
      livenessProbe:
        httpGet:
          path: /api/health
          port: 5000
        initialDelaySeconds: 15
        periodSeconds: 30
      readinessProbe:
        httpGet:
          path: /api/health
          port: 5000
        initialDelaySeconds: 5
        periodSeconds: 10
```

3.3.3 Frontend deployment

In the frontend deployment on the k8s cluster my manifest file simply just has the frontend image running and the parameters passed for its configuration, along with health checks

```yaml
containers:
  - name: frontend
    image: arcanevaultacr.azurecr.io/frontend:latest
    ports:
      - containerPort: 80
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 200m
        memory: 256Mi
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 30
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
```

3.3.4 WireGuard deployment

For the Wireguard container we have similar manifest, but it sets up the Wireguard IP tables and NAT rules, port forwarding to the on prem DB.

```
containers:
  - name: wireguard
    image: linuxserver/wireguard:latest
    securityContext:
      capabilities:
        add: ["NET_ADMIN", "SYS_MODULE"]
    env:
      - name: PUID
        value: "1000"
      - name: PGID
        value: "1000"
    ports:
      - containerPort: 51820
        protocol: UDP
      - containerPort: 5432
        protocol: TCP
      - containerPort: 10022
        protocol: TCP
      - containerPort: 10122
        protocol: TCP
      - containerPort: 10222
        protocol: TCP
    volumeMounts:
      - name: wg-config
        mountPath: /config/wg_confs/wg0.conf
        subPath: wg0.conf
      - name: lib-modules
        mountPath: /lib/modules
        readOnly: true
```

This is the iptables for the DB Proxmox

```
# Service: PostgreSQL access to DB VM (10.0.10.101)
apiVersion: v1
kind: Service
metadata:
  name: wireguard-db
  namespace: arcane-vault
spec:
  selector:
    app: arcane-vault
    component: wireguard
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432
      protocol: TCP
  type: ClusterIP
```

Lastly at the end it created the wg0.conf file with the following configurations.

```
apiVersion: v1
kind: Secret
metadata:
  name: wireguard-config
  namespace: arcane-vault
type: Opaque
stringData:
  wg0.conf: |
    [Interface]
    Address = 10.100.0.1/24
    ListenPort = 51827
    PrivateKey = 8CCRgwPPR3WV5N4Qvilii2WZZ2/LBMgsmVLlJ1r0N0Y=
    PostUp = iptables -t nat -A PREROUTING -p tcp --dport 5432 -j DNAT --to-destination 10.0.10.101:5432; iptables -t nat -A PREROUTING -p tcp --dport 10022 -j DNAT --to-destination 10.0.10.100:22
    PostDown = iptables -t nat -D PREROUTING -p tcp --dport 5432 -j DNAT --to-destination 10.0.10.101:5432; iptables -t nat -D PREROUTING -p tcp --dport 10022 -j DNAT --to-destination 10.0.10.100:

    [Peer]
    PublicKey = CDf66USHcT/RBsPMRo/wNhShEbN0ZhERdbHJkbUL2i8=
    AllowedIPs = 10.0.10.0/24, 10.100.0.0/24
    Endpoint = 158.64.39.210:51827
    PersistentKeepalive = 25
```

3.3.5 Ingress and TLS

For the Kubernetes ingress we use annotations to pass configuration arguments to the Kubernetes ingress to limit the rate of access to use the certificate we issued to allow access through certain methods and to enable TLS.

```yaml
kind: Ingress
metadata:
  name: arcane-vault-ingress
  namespace: arcane-vault
  annotations:
    cert-manager.io/cluster-issuer: selfsigned-issuer
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "5m"
    nginx.ingress.kubernetes.io/limit-rps: "100"
    nginx.ingress.kubernetes.io/limit-burst-multiplier: "10"
    nginx.ingress.kubernetes.io/enable-cors: "true"
    nginx.ingress.kubernetes.io/cors-allow-origin: "https://20.199.63.116"
    nginx.ingress.kubernetes.io/cors-allow-methods: "GET, POST, PUT, DELETE, OPTIONS"
    nginx.ingress.kubernetes.io/cors-allow-headers: "Authorization, Content-Type"
    nginx.ingress.kubernetes.io/cors-allow-credentials: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - arcanevault.example.com
      secretName: arcane-vault-tls
```

3.4 Horizontal Pod Autoscaler

For the HPA this manifest controls the pod autoscaler settings configuring when the pods should be scaled and their limits.

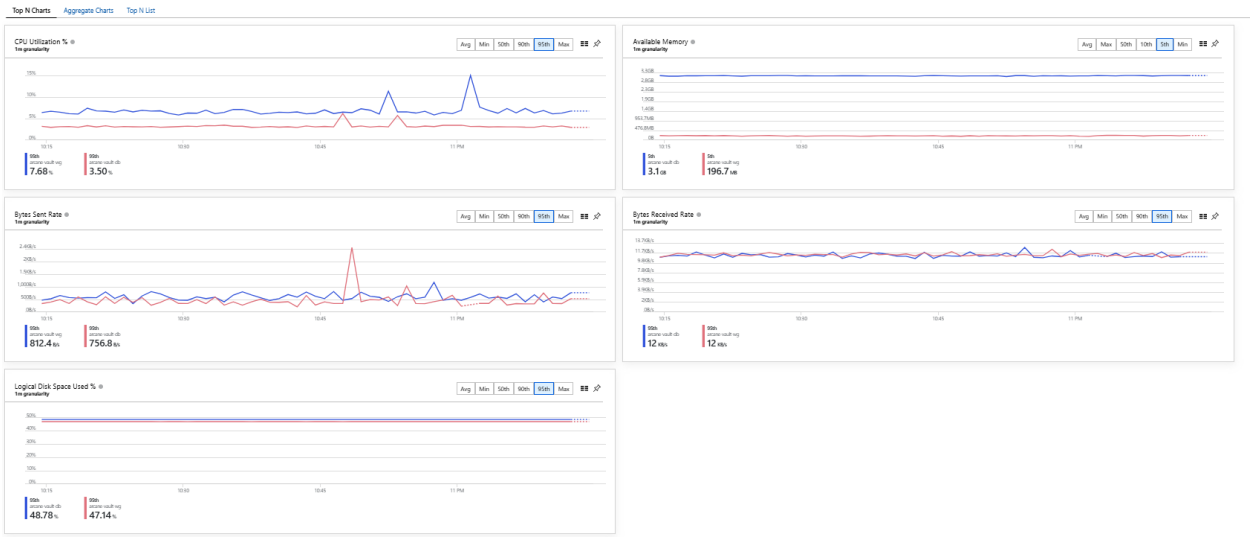
So for the manifest it will scale the backend pod based on CPU utilization upwards of 70% and memory upwards of 80%.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: arcane-vault
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 1
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

4. Monitoring and Costs

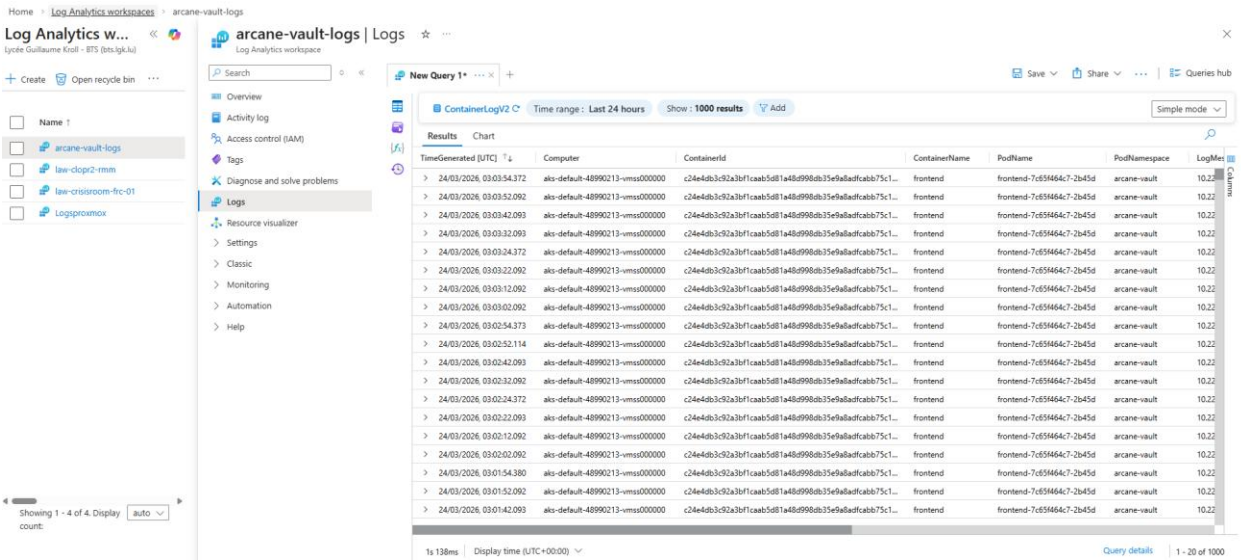
4.1 Azure Arc monitoring

For my monitoring of the project resources I have everything centralized on azure, so this means my virtual machines are onboarded onto azure with the azure insights enabled on all premise virtual machines so that I can collect metrics from the agents and monitor the health of my virtual machines over the azure monitor service on the portal.



4.2 Azure Log Analytics

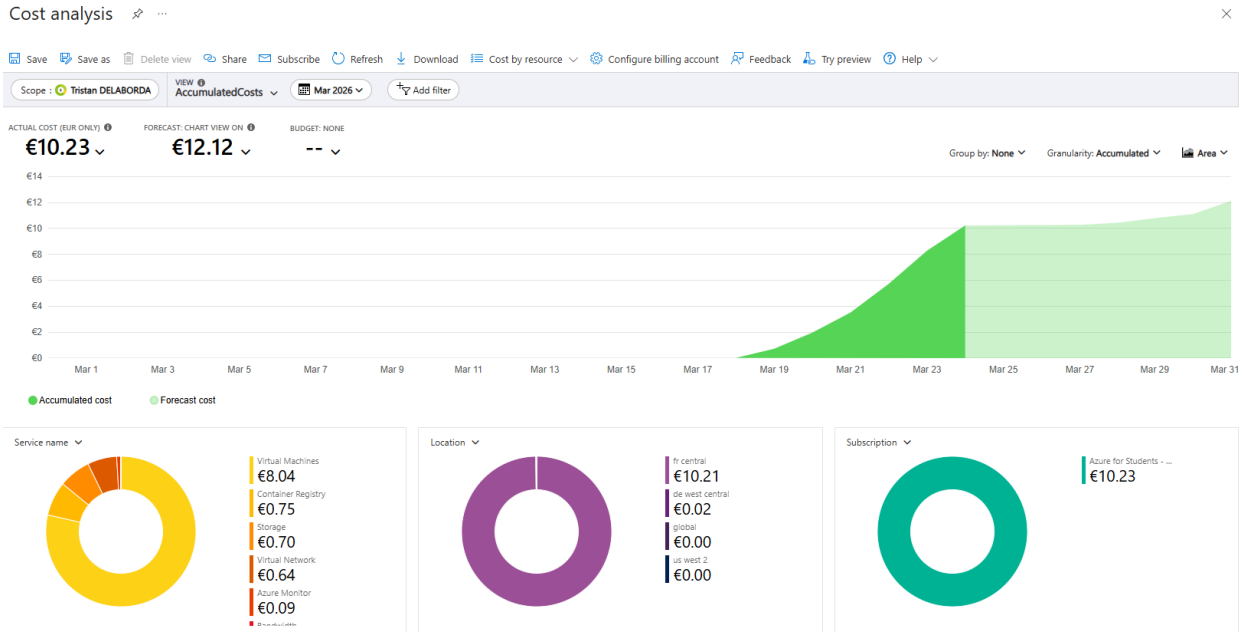
For the log analytics I can check in the portal in the log analytics workspace what my logs associated with all my resources and I can query the logs:



In the log analytics workspace I can select a log branch to query, and I can select what to query for, and it will output the queried logs as shown above.

4.3 Cost Analysis

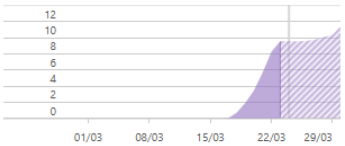
For the cost tracking and analysis I have implemented the prerequisite budget alerts to track and maintain higher level of awareness in regard to the cost of my operations. On the azure portal in the Cost and Billing service I can also see the entire cost of my deployments and over the last week of which most of my deployment has been running the overall cost has not been very high.



This are the metrics shown for my subscription:

Subscription ID	: 720f1b41-2422-4013-a9ce-1ff53e7624cc	Subscription name	: Azure for Students - deltr691
Directory	: Lycée Guillaume Kroll - BTS (bts.lgk.lu)	My role	: Owner
Status	: Active	Plan	: Azure Plan
Parent management group	: 5338e0bc-c9a3-47ba-8dd5-4cf8ae191d99	Secure Score	: Not available

Spending rate and forecast



Current cost €9.58

Forecast €11.47

View details

Costs by resource

aks-default-31250830-vmss €2.66

aks-default-48990213-vmss €2.35

aks-default-41110700-vmss €1.16

Others €3.41

View details

Top free services by usage

Used within limit 8

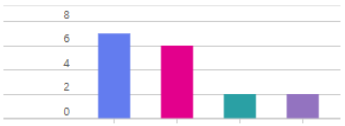
Limit exceeded 0

Unused services 50

Service	Usage
Load Balancer, Standard, Data Processed	12.46 / 15 (1 GB)
Load Balancer, Standard, Included LB Rules and Outbound Rules	121.16 / 750 (1 Hour)
Storage, Premium Page Blob, P6 Disks	0.06 / 2.2 (1/Month)
Storage, Tiered Block Blob, Hot Read Operations	0.02 / 2 (10K)
Storage, Tiered Block Blob, Hot LRS Write Operations	0.01 / 1 (10K)
Networking, Data Transfer Out (GB)	0.02 / 15 (1 GB)

View all free services

Top products by number of resources



1/2

datacollectionrules 7

prometheusrulegroups 6

View resources

Azure Defender coverage

Azure Defender is not enabled for this subscription

Upgrade coverage

5. Feature implementations

The following table summarises the implementations of all features from the original project proposal, along with any additional features.

5.1 Infrastructure features

Feature	Status	Notes
Kubernetes orchestration (AKS)	**Implemented**	AKS with auto-scaling node pool
Docker containers	**Implemented**	Multi-stage builds, health checks
Terraform IaaC	**Implemented**	Azure + Proxmox provisioning
Auto-scalable resources (HPA)	**Implemented**	CPU 70%, memory 80% targets
Hybrid cloud (Azure + on-prem)	**Implemented**	AKS + Proxmox via WireGuard
On-premises database	**Implemented**	PostgreSQL 16 on Proxmox VM
Ansible automation	**Implemented**	2-VM configuration playbook
State preservation / fault tolerance	**Implemented**	K8s probes, HPA, persistent volumes
Monitoring (Azure Arc)	**Implemented**	Replaced Prometheus with Azure Arc
Redis caching	**Implemented**	Added to AKS cluster

5.2 Security features

Feature	Status	Notes
Network segregation (NetworkPolicies)	**Implemented**	Backend + frontend policies
Container security / least privilege	**Implemented**	Non-root, resource limits
Entra ID for IAM	**Implemented**	Full MSAL + JWT validation
Azure RBAC on AKS	**Implemented**	Entra ID tenant binding
TLS / encryption in transit	**Implemented**	cert-manager + WireGuard
Firewall rules (UFW)	**Implemented**	All VMs, deny-by-default

Feature	Status	Notes
Rate limiting	**Implemented**	Nginx + Express layers
Secrets management	**Partial**	K8s Secrets + Ansible Vault
Data encryption at rest	**Partial**	Azure-managed disk encryption only
Multi-factor authentication	**Not implemented**	Excluded from scope
Web application firewall (WAF)	**Not implemented**	Excluded from scope

5.3 Application features

Feature	Status	Notes
Character sheets with editable fields	**Implemented**	Full CRUD with wizard
Validation checks	**Implemented**	Frontend + backend + DB constraints
Data persistence (PostgreSQL)	**Implemented**	Users + characters tables
WebSocket real-time features	**Not implemented**	Not in current codebase, but wasn't explicitly specified

5.4 Additions beyond original proposal

Feature	Status	Notes
Public Vault (community gallery)	**Implemented**	New feature
WireGuard VPN tunnel	**Implemented**	New feature
Azure Arc integration	**Implemented**	New feature
Terraform state backend bootstrap	**Implemented**	New feature
Full automated deploy script	**Implemented**	New feature
D&D reference data	**Implemented**	New feature

Feature	Status	Notes
Nginx ingress with rate limiting	**Implemented**	New feature

6. Challenges and lessons learned

6.1 Kubernetes

For the first challenge I outline my struggles with Kubernetes and how my initial approach to this task was flawed. From the beginning of the project management phase I had only allocated 1 sprint to the setup and configuration of the Kubernetes cluster thinking this would be enough. I was mistaken, quickly the Kubernetes task during the last week of my project submission became more of a daunting task than anticipated.

Starting from the initial research and development leading into the sprint I started working on the Kubernetes cluster last week Wednesday this would be about 6 days before submission deadline. The core functionality of Kubernetes was still fully realized on Friday that week, which would be 4 days before submission deadline. This was a frightening mix that wasn't planned and dealt with poorly thus sending me into a hopeless pit of despair, draining my morale. During the last week it also made it additionally challenging since there was no room left in my project planning for this hinderance, so I had to tackle the challenge head on and at full risk of losing project time on other more pressing tasks and in some cases even dropping project deliverables to make room for this foresight.

The Kubernetes was furthermore challenging for this project because my primary goal was automation and along with all the deliverables, I had a lot of tasks piling up on my Kubernetes stack during this altercation.

Core Problems

- Intial mistakes with not researching proper deployments lead to use of B series VMs for nodes causing a lot of issues.
- Initial implementation was very bloated and saturated 229% of the VM resources.
- Stripping Calico plugin to save on resources.
- Lack of testing before attempting full deployment.
- Misconfigurations that seemed small expanded into big problems, Such as Wireguard running at privileged mode: true crashing my entire node due to applied routes to entire node.
- Use of rollout command without proper research into what it does lead to my replication sets lingering and creating infinity new pods.
- HPA and replication sets limited to 1 min and 1 max prevented scheduler from recreating pods before old pods have been terminated.

Key takeaways

- Poor project planning at the cause of unchecked scope.

- Little to no room for leverage when dealing with mis planned or time-consuming events such as this one.
- Erosion of self-morale and trust in my own abilities.
- Diminished quality of work.
- Opportunity cost due to dropping other features that could have been delivered upon.

Root cause analysis

- Lack of clearly defined checkpoints: Unsure where to progress, on what to progress and on how to progress most effectively lead to goal posts always being pushed back.
- Partial fear of asking questions in the initial proposal phase of the submission: Lack of insight from lecturers that could have aided me with knowledge on how to schedule this task.

Self-reflection

- Better preparation needed when dealing with unfamiliar tasks and objectives
- Ask for clarity and even verification of uncertain tasks when planning them out during the mid-week meetings for insight on if I am still on track.
- Aim to be more prepared in case of disasters such as this, set goals for such cases and leave space to breathe.
- Study and research how a current deployment of such a task would look like to get a general idea of what I am expected to deliver upon.

6.2 Proxmox VM deployment using Cloud-init

During the second week of the sprints, I attempted to deploy my virtual machines infrastructure using terraform. I had looked into deploying the VMs using a cloud-init template and initially generated a terraform script that would deploy a simple virtual machine for all my resources including my Wireguard a Postgres SQL database and setup the firewall rules to only allow access from the management node I was currently on. Initial implementations of this went well enough, however the virtual machine would not get its Ips correctly defined. I had initially created a Virtual machine to convert to a template to use as the baselines cloud-init cloned virtual machine template. Once the Virtual machine was created I then added the cloud init drive to the virtual machine and had converted it to a template. This was my first mistake but at the time this was not clear to me.

The Virtual machine would take 15 minutes to deploy as the cloud-init configured NIC would take that long to time out, erroring out my terraform script mid deployment. This Led to extensive research, time consuming tasks, redeployment trials, testing parameters

to diagnose the problem that was not a problem the configuration on the cloud-init was collect but never being applied to the machine upon creation. My mindset was to fix the problem and I spend very long allocating lots of time and effort into fixing this issue, but to no avail. After help from my classmates and professors I was become aware of a mis configuration error in my design, in which I had added the cloud-init drive prior to creation of the template which was the wrong time to assign this, and as even mentioned in one of our external expert meetings we were told that we had to assign the drive before installation of the operating system. This had fixed my misconfiguration error but not before the time and resources had already been wasted making progression less effort.

Core Problems

- Initial hasty misconfiguration of Cloud-init drive
- Initial flawed sequencing of process pipeline
- Lack of knowledge on what cloud init requirements are

Root cause analysis

- Cloud-init relies on being present before OS initial first boot to inject configurations like static Ips and Dns.
- Terraform output waited for DCHP timeout since NIC was never assigned valid Ips from misconfiguration.

Key takeaways

- Spending excess time debugging irrelevant problems (Like draining water on a sinking ship rather than patching the hole)
- Failing to see the bigger picture during the debugging.
- Failure to consider other providers as options.
- Attaching cloud-init drive too late due to lack of awareness.
- Diagnosing a problem without understanding it.

Self-reflection

- Lack of awareness to the problem and also in seeking help
- Lack of attention to already familiar problem.
- Lack of external awareness: I focused too much on the internal Terraform and Virtual machine issues I failed to take a step back and recognize other determining factors.
- Lack of experience in this domain but with this problem now this reinforced that infrastructure issues are often caused in preparation steps.
- Too many assumptions lead me down a more difficult path that could have been easily avoided.

7. Conclusion

Overall, I think the project was a success, I have learned many new skills during this experience. As for the management of the project I aim to take that aspect more seriously in future challenged and for the challenges with the hands-on approach to projects and the fragility of how important we well managed project can be have been solidified as permanent markers for what should be prioritized in these sorts of assignments. I think that the growth in the field of experiencing cloud computing technologies speaks for itself, and along with the key achievements in setting up the complexities behind this implementation of a hybrid cloud deployment that focuses primarily on automation.

As the project drew to a close the sure signed gravity of the situation-imposed stress on me, but in the grand picture of the journey I took to get here, In the end of it all I would still say that I had fun attempting something new and challenging.

8. References and appendices

8.1 Appendix A: Configuration files Refernces

https://registry.terraform.io/providers/bpg/proxmox/latest/docs

https://registry.terraform.io/providers/hashicorp/azuread/latest/docs

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

https://docs.docker.com/build/building/multi-stage/

https://www.wireguard.com/quickstart/

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/kubernetes_cluster

8.1 Appendix B: AI Prompts

Terraform & Infrastructure

- My terraform script is erroring, could you help me with that?
- Could you help me diagnose my Ansible?
- How do I fix this issue inside my deployment files so that next time I deploy it already has the permissions? (Special Case)
- How would I go about redeploying the new TF to make sure this error doesn't happen again?
- Based on that solution, what changes would I have to do to the Ansible deployment?

AKS & Kubernetes

- What are CPU credits? (B-series VM explanation)
- Before I apply my WireGuard, how do I fix any issues within the YAML?
- In my Terraform, where would be the most ideal place for a Redis caching service?
- How would I go about re-deploying Redis into my Kubernetes cluster?
-

Networking & WireGuard & Frontend

- How do I test connectivity in my VMs and WireGuard?
- Is it possible to change my WireGuard and bind it to the secondary network interface?

- Could you explain why the sign-in opens a Microsoft sign-in page that errors out?